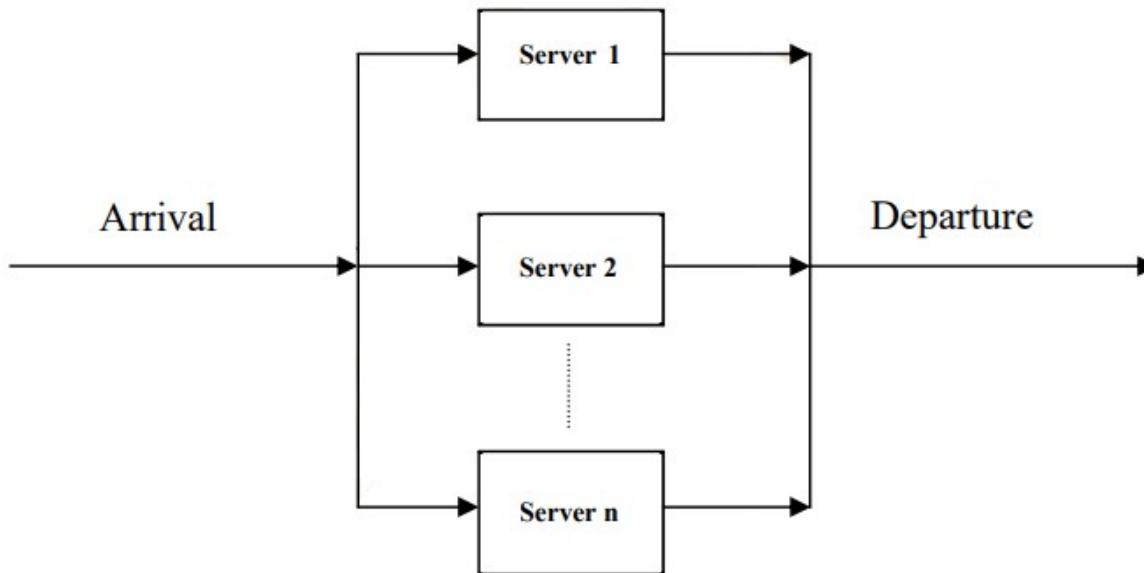


CodeCrunch

Lab #2

We would like to simulate how customers are being served by servers in a shop. Specifically, a customer that arrives will look for the first available server so as to be served for some amount of time. On seeing that all servers are busy, a new arriving customer will just leave.

The following depicts a schematic of the simulation:



- There are n servers;
- Each server can serve one customer at a time;
- Each customer that is served has a service time (time taken to serve the customer);
- When a customer arrives:
 - the servers are scanned from 1 to n to find the first one that is available (not serving any customer) and gets served immediately.
 - if all servers are serving customers, then the customer just leaves.

As an example, you are given an input of two servers and three customers, followed by the the integer identifier and floating-point arrival times of the customers,

```
2 3
1 0.500
2 0.600
3 0.700
```

Assuming that the service time to service each customer is constant at 1.0, a simulation can be performed with the following output that shows how each customer is served.

```
customer 1 arrives
customer 1 served by server 1
customer 2 arrives
customer 2 served by server 2
customer 3 arrives
customer 3 leaves
```

This task comprises a number of levels. You are required to complete ALL levels.

Level 0

Let us start by getting acquainted with the `InfList` class which have been provided for you.

The `InfList` class is an implementation of an infinite list that works in the same way as Java's `Stream` but without the restriction of a single use. This is to avoid storing the stream as a list, in order to stream repeatedly.

Also note that `InfList` is dependent on the `Maybe` class, which is explained in detail later.

The programs [InfList.java](#) and [Maybe.java](#) are given. Specifically, `InfList.java` includes [javadoc comments](#). To automatically generate HTML documentation from the comments, issue the command:

```
$ javadoc -d doc Maybe.java InfList.java
```

You may then navigate through the documentation from `allclasses-index.html` found in the `doc` directory.

```
$ jshell Maybe.java InfList.java
| Welcome to JShell -- Version 21.0.2
| For an introduction type: /help intro

jshell> InfList<Integer> ifl = InfList.iterate(1, x -> x + 1).limit(3)
ifl ==> InfList$.

jshell> ifl.forEach(x -> System.out.println(x))
1
2
3

jshell> ifl.reduce(0, (x, y) -> x + y)
$. ==> 6
```

Level 1

Write the `Customer` class with a constructor that takes in an identifier of type `int` and an arrival time of type `double`.

```
jshell> new Customer(1, 1.0)
$. ==> customer 1

jshell> List<Customer> customers = List.of(new Customer(1, 1.0), new Customer(2, 2.0))
customers ==> [customer 1, customer 2]

jshell> InfList<Customer> ifl = InfList.iterate(0, x -> x + 1).limit(customers.size()).map(x -> customers.get(x))
ifl ==> InfList$.

jshell> ifl.forEach(x -> System.out.println(x))
customer 1
customer 2
```

Include the method `canBeServed` that takes in a `time`, and returns `true` when the customer arrives exactly or later than `time`; any earlier and the customer would have left.

```
jshell> ifl.map(c -> c.canBeServed(2.0)).
...> forEach(x -> System.out.println(x))
false
true
```

Also include the method `serveTill` that takes in a service time, and returns the time of the end of service.

```
jshell> ifl.map(c -> c.serveTill(1.0)).
...> forEach(x -> System.out.println(x))
2.0
3.0
```

Level 2

Write the `Server` class with a constructor that takes in an integer identifier of the server. Each new server is ready to serve the next customer.

```
jshell> new Server(1)
$.. ==> server 1
```

Write a method `serve` that allows a server to serve a customer for a service time specified. As an example, the following creates a new server #1 to serve customer #10 which arrives at 1.0, for a service time of 2.0.

```
jshell> new Server(1).serve(new Customer(10, 1.0), 2.0)
$.. ==> server 1
```

Server #1 is available to serve from time 0.0, starts serving customer #10 at 1.0 and ends the service at 3.0. The server will only be available to serve another customer from time 3.0 onwards.

Write a method `canServe` to determine if a server can serve a customer. As an example, the following creates a new server #1 and determines if he/she can serve customer #1 which arrives at 1.0.

```
jshell> new Server(1).canServe(new Customer(1, 1.0))
$.. ==> true
```

In the following, server #1 serves customer #1 at 1.0 and becomes available at 3.0.

```
jshell> new Server(1).serve(new Customer(1, 1.0), 2.0).canServe(new Customer(2, 3.0))
$.. ==> true
```

You can also use the infinite list to test your implementation.

```
jshell> ifl.map(c -> new Server(1).canServe(c)).
...> forEach(x -> System.out.println(x))
true
true

jshell> ifl.map(c -> new Server(1).serve(new Customer(10, 1.0), 1.0).canServe(c)).
...> forEach(x -> System.out.println(x))
false
true

jshell> ifl.map(c -> new Server(1).serve(new Customer(10, 1.0), 2.0).canServe(c)).
...> forEach(x -> System.out.println(x))
false
false
```

Level 3

Write the class `Shop` with a constructor that takes in the number of servers so as to encapsulate all the servers. Each server is identified by a running number starting from 1. Note that a shop can have no servers.

```
jshell> new Shop(2)
$.. ==> Shop:<server 1><server 2>
```

```
jshell> new Shop(0)
$.. ==> Shop:
```

Write a method `findServer` that takes in a customer and finds the first server in the shop that can serve the customer. Since a shop can be empty (or all servers are busy to serve the customer), the method should return an `Maybe<Server>`.

```
jshell> new Shop(2).findServer(new Customer(1, 1.0))
$.. ==> Maybe[server 1]
```

```
jshell> new Shop(0).findServer(new Customer(1, 1.0))
$.. ==> Maybe.empty
```

Hint: the `findFirst` method in `InfList` returns an object of the [Maybe](#) class (similar to `findFirst` method in `Stream` returning an object of Java's `Optional` class). The sample run below illustrates how `Maybe` is used with the `map` and `filter` methods. The `ifPresent` method can be used to output the value encapsulated within the `Maybe` (much like `forEach` in `InfList` OR `Stream`).

```
jshell> Maybe.of(1)
$.. ==> Maybe[1]

jshell> Maybe.of(1).map(x -> x + 1)
$.. ==> Maybe[2]

jshell> Maybe.of(1).map(x -> x + 1).ifPresent(x -> System.out.println(x))
2

jshell> Maybe.of(1).map(x -> x + 1).filter(x -> x == 0)
$.. ==> Maybe.empty

jshell> Maybe.of(1).map(x -> x + 1).filter(x -> x == 0).ifPresent(x -> System.out.println(x))
```

Write a method `update` that takes in a server as argument and updates the corresponding server (with same identifier) in the shop with this new one. You may add more methods in the `Server` class to check for the same servers.

```
jshell> new Shop(2).update(new Server(1).serve(new Customer(1, 1.0), 1.0))
$.. ==> Shop:<server 1><server 2>

jshell> new Shop(2).update(new Server(1).serve(new Customer(1, 1.0), 1.0)).
...> findServer(new Customer(2, 1.0))
$.. ==> Maybe[server 2]
```

Notice in the last example that after updating server #1 to serve a customer at 1.0 until 2.0, the available server to serve customer #2 arriving at 1.0 is now server #2. Here is an example when both servers are busy.

```
jshell> new Shop(2).update(new Server(1).serve(new Customer(1, 1.0), 1.0)).
...> update(new Server(2).serve(new Customer(2, 1.0), 1.0)).
...> findServer(new Customer(3, 1.0))
$.. ==> Maybe.empty
```

Level 4

Study the following [Simulator](#) class which has been provided for you.

```
class Simulator {
    private final int numOfServers;
    private final int numOfCustomers;
    private final InfList<Pair<Integer,Double>> arrivals;
    private final double serviceTime;

    Simulator(int numOfServers, int numOfCustomers,
              InfList<Pair<Integer,Double>> arrivals, double serviceTime) {
        this.numOfServers = numOfServers;
        this.numOfCustomers = numOfCustomers;
        this.arrivals = arrivals;
        this.serviceTime = serviceTime;
    }

    State run() {
        State init = new State(new Shop(numOfServers));
        return arrivals.map(x -> new Customer(x.t(), x.u()))
            .reduce(init, (s, c) -> s.next(c));
    }
}
```

For each simulation, a `Simulator` object is created with a constructor that takes in the number of servers, the number of customers, all paired values comprising the customer id and arrival time, and a fixed service time. For simplicity, you may assume from this point onwards that service time for each customer is strictly 1.0.

The generic [Pair](#) record has been provided for you to create a pair of values of possibly different types.

```
record Pair<T,U>(T t, U u) {}
```

Here is how you can create a pair of values comprising an integer and a string.

```
jshell> new Pair<Integer,String>(1, "one")
$.. ==> Pair[t=1, u=one]
```

```
jshell> new Pair<Integer,String>(1, "one").t()
$.. ==> 1
```

```
jshell> new Pair<Integer,String>(1, "one").u()
$.. ==> "one"
```

The `run` method in the `Simulator` class starts with an initial state of a shop of free servers, transitioning from one state to the next via the `next` method that takes in a customer and returns the next state. Moreover, the string representation of a state gives the simulation output for that most recent state only.

Here are some examples,

```
jshell> new State(new Shop(2))
$.. ==>
```

```
jshell> new State(new Shop(2)).
...> next(new Customer(1, 1.0))
$.. ==>
customer 1 arrives
customer 1 served by server 1
```

```
jshell> new State(new Shop(2)).
...> next(new Customer(1, 1.0)).
...> next(new Customer(2, 2.0))
$.. ==>
customer 1 arrives
customer 1 served by server 1
customer 2 arrives
customer 2 served by server 1
```

```
jshell> new State(new Shop(2)).
...> next(new Customer(1, 1.0)).
...> next(new Customer(2, 1.5))
$.. ==>
customer 1 arrives
customer 1 served by server 1
customer 2 arrives
customer 2 served by server 2
```

```
jshell> new State(new Shop(3)).
...> next(new Customer(1, 0.5)).
...> next(new Customer(2, 0.6)).
...> next(new Customer(3, 0.7))
$.. ==>
customer 1 arrives
customer 1 served by server 1
customer 2 arrives
customer 2 served by server 2
customer 3 arrives
customer 3 served by server 3
```

Your task now is to write the `State` class.

Hint: another useful `Maybe` method is the `orElse` method which returns the value within the `Maybe` if it is present, or some default value otherwise.

```
jshell> Maybe.of(1).map(x -> x + 1).filter(x -> x == 2).orElse(-1)
$.. ==> 2
```

```
jshell> Maybe.of(1).map(x -> x + 1).filter(x -> x == 0).orElse(-1)
```

```
$.. ==> -1
```

Level 5

The [Main](#) program has been provided for you.

```
import java.util.Scanner;

static final double SERVICE_TIME = 1.0;

void main() {
    Scanner sc = new Scanner(System.in);
    int numOfServers = sc.nextInt();
    int numOfCustomers = sc.nextInt();

    sc.nextLine(); // removes trailing newline
    InfList<Pair<Integer,Double>> arrivals = InfList.iterate(1, x -> x + 1)
        .limit(numOfCustomers)
        .map(x -> new Pair<Integer,Double>(sc.nextInt(), sc.nextDouble()));

    State state = new Simulator(numOfServers, numOfCustomers, arrivals, SERVICE_TIME)
        .run();
    System.out.println(state);
}
```

This is merely a verification step. Compile your program together with the given Main class.

Sample runs of the program is given below. You should create your own input files using vim. You also need not worry about extra blank lines in the output as these will be ignored.

```
$ javac --release 21 --enable-preview Main.java
Note: Main.java uses preview features of Java SE 21.
Note: Recompile with -Xlint:preview for details.
```

```
$ cat 1.in
3 3
1 0.500
2 0.600
3 0.700
```

```
$ cat 1.in | java --enable-preview Main
```

```
customer 1 arrives
customer 1 served by server 1
customer 2 arrives
customer 2 served by server 2
customer 3 arrives
customer 3 served by server 3
```

```
$ cat 2.in
3 6
1 0.500
2 0.600
3 0.700
4 1.500
5 1.600
6 1.700
```

```
$ cat 2.in | java Main
```

```
customer 1 arrives
customer 1 served by server 1
customer 2 arrives
customer 2 served by server 2
customer 3 arrives
customer 3 served by server 3
customer 4 arrives
customer 4 served by server 1
```

```
customer 5 arrives
customer 5 served by server 2
customer 6 arrives
customer 6 served by server 3
```

```
$ cat 3.in
2 3
1 0.500
2 0.600
3 0.700
```

```
$ cat 3.in | java --enable-preview Main
```

```
customer 1 arrives
customer 1 served by server 1
customer 2 arrives
customer 2 served by server 2
customer 3 arrives
customer 3 leaves
```

```
$ cat 4.in
2 3
1 0.500
2 0.600
3 1.500
```

```
$ cat 4.in | java --enable-preview Main
```

```
customer 1 arrives
customer 1 served by server 1
customer 2 arrives
customer 2 served by server 2
customer 3 arrives
customer 3 served by server 1
```

It is interesting to note that Unix also makes use of pipelining to pipe the input of a file (e.g. `cat 1.in`) via the pipe operator `|` to the `java` command.

Your Files:

[SUBMIT](#) (only .java, .c, .cpp, .h, .jsh, and .py extensions allowed)

To submit multiple files, click on the Browse button, then select one or more files. The selected file(s) will be added to the upload queue. You can repeat this step to add more files. Check that you have all the files needed for your submission. Then click on the Submit button to upload your submission.